



푸딩툰(React + Next.js) 리뉴얼

1. 프로젝트 개요

- **프로젝트명:** PuddingToon (웹툰 서비스 플랫폼) 리뉴얼
- **개발 기간:** 2025.11 ~ 2025.12
- **수행 방식:** 애자일(Agile) 기반 협업 / Git 기반 협업
- **팀 구성:** 프론트 3인 / 후임 1명 온보딩·업무 관리
- **담당 범위:** 플레이어 기능 제외, **사이트 전반 프론트엔드 구조 설계 및 퍼블리싱~개발 ~QA 대응**
- **배포 현황:** 리뉴얼 서비스 운영 및 안정화 진행 중
- **기술 스택:** Next.js(15), React(19), TypeScript(5), Zustand, TanStack Query, Axios, TailwindCSS(4), Sass, Radix UI/Headless UI/shadcn-ui, React Hook Form, Zod, crypto-js, date-fns, Tone.js, (에디터) Lexical 기반 전환

2. 요구사항 및 나의 역할

프로젝트 개요

기존 Vue 기반 서비스를 React + Next.js(App Router)로 이전하며, 정보 구조/페이지 구성/UX를 대폭 개편한 리뉴얼 프로젝트

요구사항 요약

- 기존 Vue 서비스 → **React/Next 기반으로 마이그레이션** 및 구조 재정비
- 리뉴얼 범위 내 페이지/기능을 **4주 내 개발 완료**
- 기획/디자인/백엔드 변경이 잦은 상황에서 **애자일 방식으로 빠르게 반영**
- QA 단계에서 이슈 관리 및 안정화, 이후 플레이어 영역에도 투입되어 버그 대응

담당 역할

- **프론트엔드 오너십(전반 책임):** 리뉴얼 범위의 구조 설계부터 퍼블리싱/개발/QA 대응까지 E2E로 리딩
- **프로젝트 구조 설계:** App Router 기반 라우팅, 도메인 단위 상태관리/모델/상수/유틸 레이어 정리 및 확장 가능한 디렉토리 구조 구축
- **협업 리딩 & 후임 관리:** 3인 팀 내 작업 분배, 코드 리뷰/가이드 제공, 후임 온보딩 및 진행 관리
- **안정화 & 운영 대응:** QA 이슈 트래킹/핫픽스, 서비스 안정화 이후 플레이어 영역 버그픽스 지원

3. 기술 스택 및 아키텍처

항목	사용 기술 / 라이브러리
프레임워크	Next.js(App Router), React, TypeScript
상태 관리	Zustand(클라이언트 상태), TanStack Query(서버 상태)
UI/CSS	TailwindCSS, Sass(SCSS), Radix UI, Headless UI, shadcn/ui
API 통신	Axios(공통 인스턴스/인터셉터)
폼/검증	React Hook Form, Zod
기타	crypto-js(본인인증), date-fns, Tone.js

아키텍처 특징

- **App Router 기반 구조 + 레거시 공존:** 신규(App Router)와 v2 레거시 영역을 분리 운영하며 점진적 마이그레이션 가능한 형태로 구성
- **도메인 중심 설계:** `stores/`, `models/`, `constants/`, `lib/` 로 분리하여 인증/유저/홈/시리즈/작가/커뮤니티 등 도메인 단위로 관리
- **콜로케이션 패턴 적용:** 페이지 단위로 `section/components/hooks/lib/types` 를 묶어 유지보수성과 작업 속도 향상
- **공통 API 레이어 표준화:** Axios 인스턴스/인터셉터를 통해 토큰 갱신, 에러 핸들링을 공통 처리하여 페이지별 중복 제거

4. 주요 기능

기능	설명
인증 시스템	일반 로그인 + 소셜 로그인(카카오/네이버/구글/애플), 콜백 라우팅 및 토큰 처리
본인인증	이니시스 본인인증 연동 및 암호화 처리(crypto-js)
전역 상태/서버 상태	Zustand로 UI/클라이언트 상태, React Query로 API 서버 상태 캐싱/재검증
공통 UI/컴포넌트	shadcn/ui 기반 공통 컴포넌트 + Radix/Headless로 접근성 고려
안정화/QA	staging 환경 QA 대응 및 운영 안정화, 이후 플레이어 버그픽스 지원

5. 문제 해결 및 기술적 도전

✓ 문제 1: Vue → React/Next 마이그레이션을 4주 내 완료해야 하는 일정 압박

- 요구사항 변경과 작업 병렬화가 필수였고, “나중에 정리”로는 절대 일정이 안 나오는 상황

→ 확장 가능한 기본 골격(라우팅/상태/모델/API 레이어)을 먼저 확정하고, 기능을 수평 확장하는 방식으로 진행.

페이지 단위 **콜로케이션 패턴**을 적용해 화면 개발 속도를 확보하고, 공통 컴포넌트(shadcn/ui)로 UI 생산성을 끌어올림.

✓ 문제 2: 에디터(글쓰기) 성능/호환 이슈

Quill 에디터가 React 버전과 충돌하며 입력 렉/비정상 동작 이슈 발생

→ **Lexical 데이터 기반 구조로 전환**하여 호환성 문제를 제거하고 안정적으로 글쓰기 플로우를 유지.

✓ 문제 3: 인증 상태 동기화 & 재검증 전략

로그인 상태는 빠르게 보여야 하고, 동시에 최신 상태를 유지해야 함(탭 전환/토큰 만료 등)

→ 인증 훅/Provider/ProtectedRoute 패턴으로 **인증 필수/선택 페이지 UX를 표준화**하고, 현재는 **TanStack Query 기반으로 인증 흐름 리팩토링**을 진행하여 서버 상태 캐싱/재검증 정책을 일관되게 가져가는 중.

✓ 문제 4: QA/운영 단계에서의 안정화 대응

리뉴얼 이후 실제 사용자 이슈가 유입되고, staging QA에서 발견되는 이슈가 연쇄적으로 발생

→ QA 이슈를 우선순위로 정리해 핫픽스/재검증 루프를 운영했고, 안정화 이후에는 **플레이어 영역에도 투입되어 버그를 빠르게 정리하며 운영 품질을 개선.**

6. 협업 및 커뮤니케이션

- 기획/디자인/백엔드와 애자일 방식으로 주기적인 싱크를 돌리며 요구사항 변경을 빠르게 반영
 - Git 기반 협업에서 `dev → staging → main` 흐름으로 **통합 테스트/QA/운영 배포 단위를 분리하여 릴리즈 안정성 확보**
 - 팀 내 후임 1명에게 작업 가이드/리뷰를 제공하며 일정과 품질을 함께 관리
-

7. 개발 결과 및 회고

결과

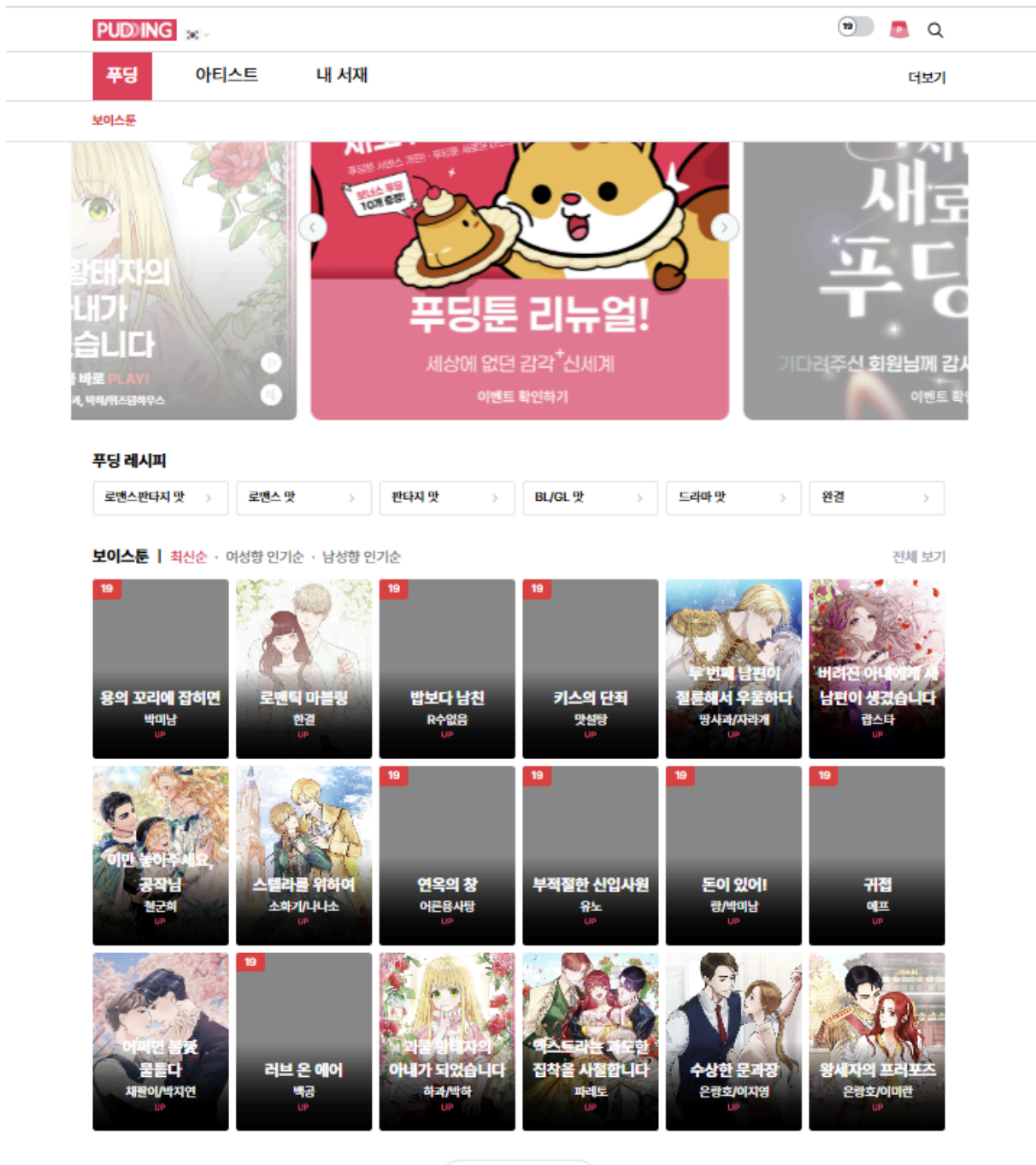
- Vue 기반 서비스의 핵심 UI/페이지를 **React + Next.js로 이전**하며 리뉴얼 범위를 일정(4주) 내 개발 완료
- 구조 설계부터 퍼블리싱/개발/QA 대응까지 전반을 리딩하며 **프론트 개발 생산성과 유지보수성 기반을 정착**
- 안정화 단계에서 QA 이슈를 관리하고 운영 품질을 개선, 이후 플레이어 영역 버그픽스까지 지원하며 서비스 안정화 기여

회고

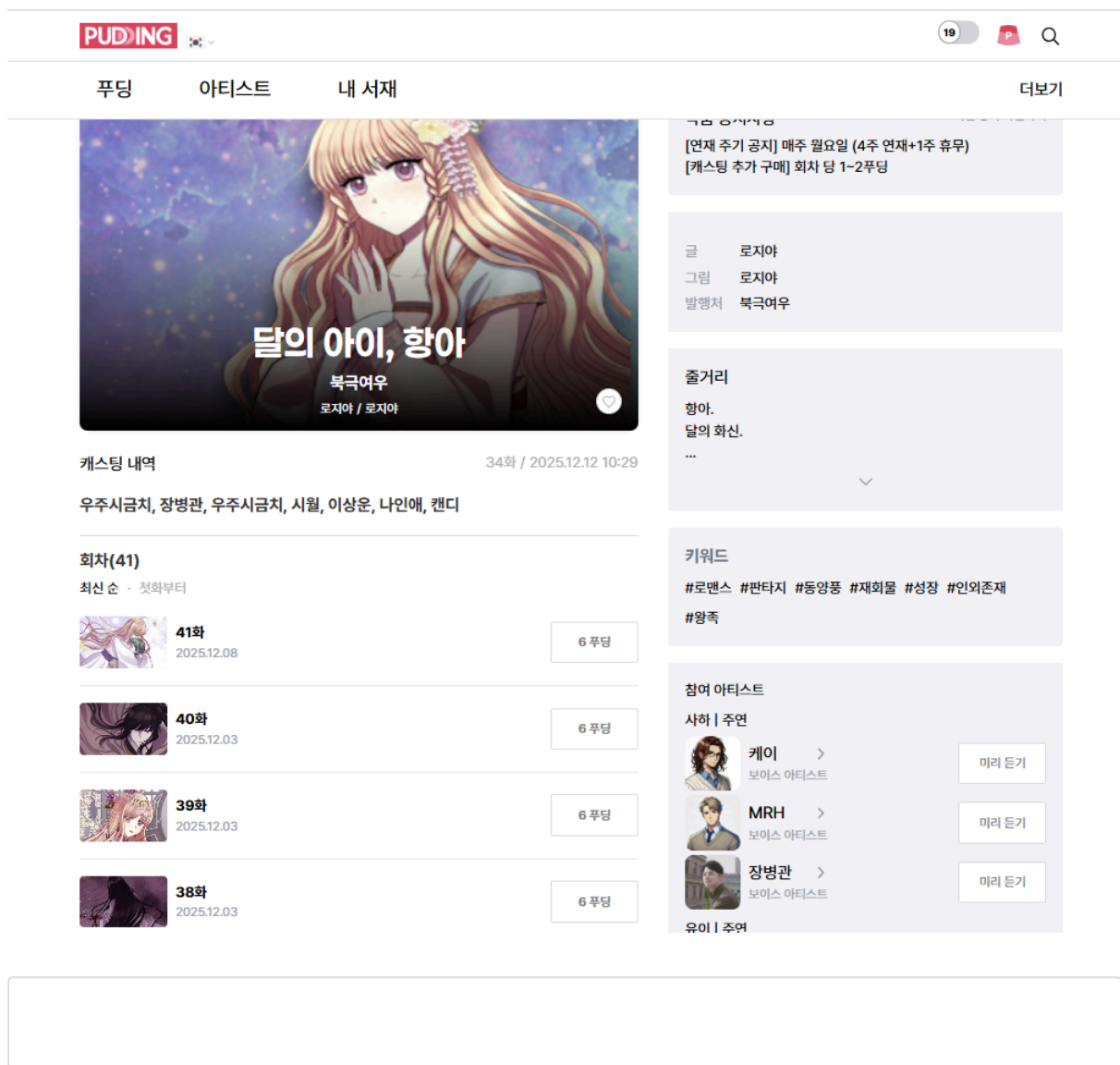
- “빠른 전환”이 필요한 상황에서 **구조(기본 골격) 선 확정 → 기능 수평 확장** 방식이 효과적이었고,
 - 리뉴얼은 개발만이 아니라 QA/운영까지 포함된다는 걸 다시 체감. 이후에는 인증/세션/에러 처리 같은 공통 정책을 더 단단히 만들어 팀 전체 비용을 줄이는 방향으로 개선 중
-

8. 스크린샷

- 앱 주요 화면별 캡처 이미지 (홈 / 기능 A / 기능 B 등)



이미지 1. 메인 페이지

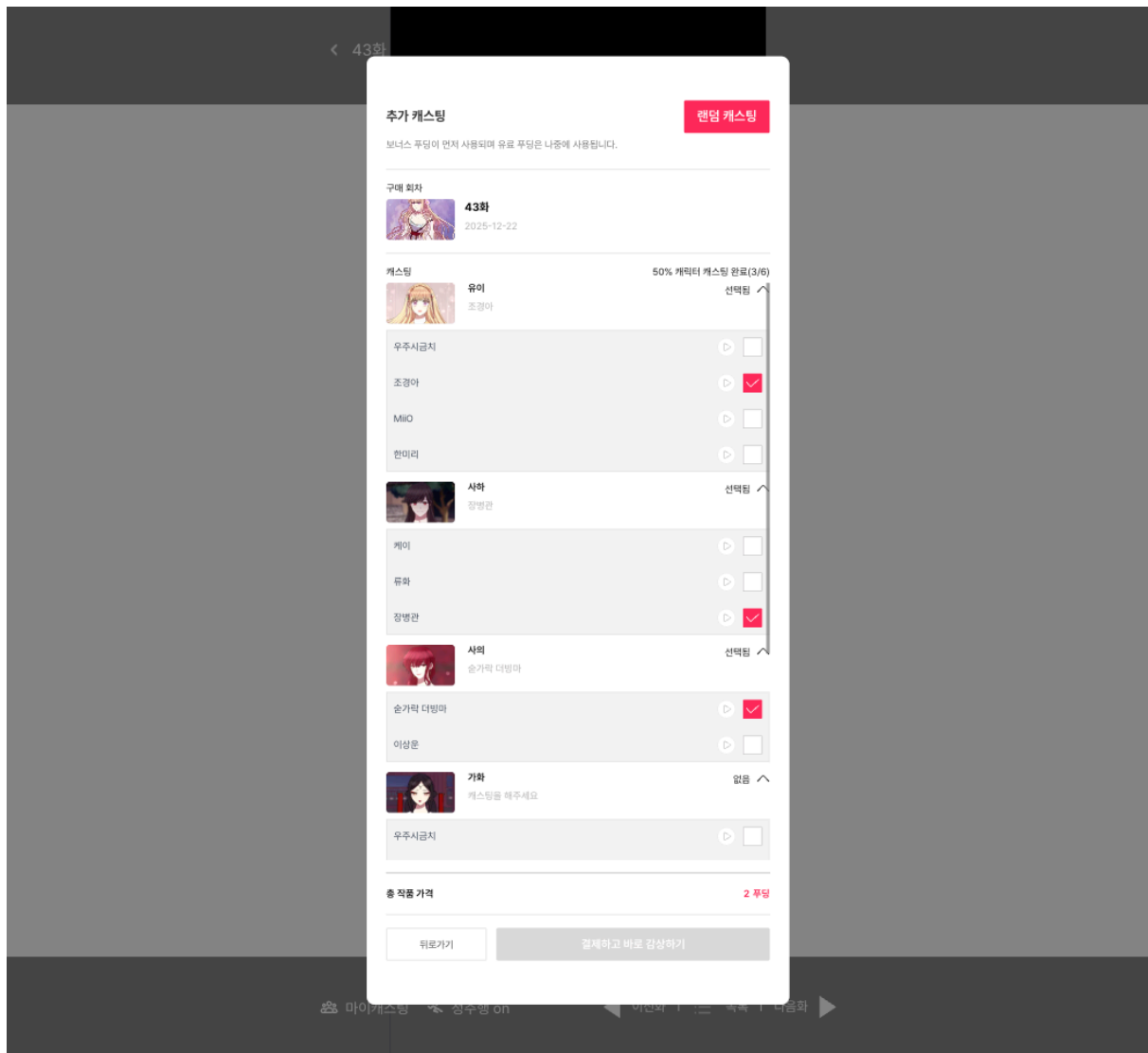


이미지 2. 작품상세 페이지

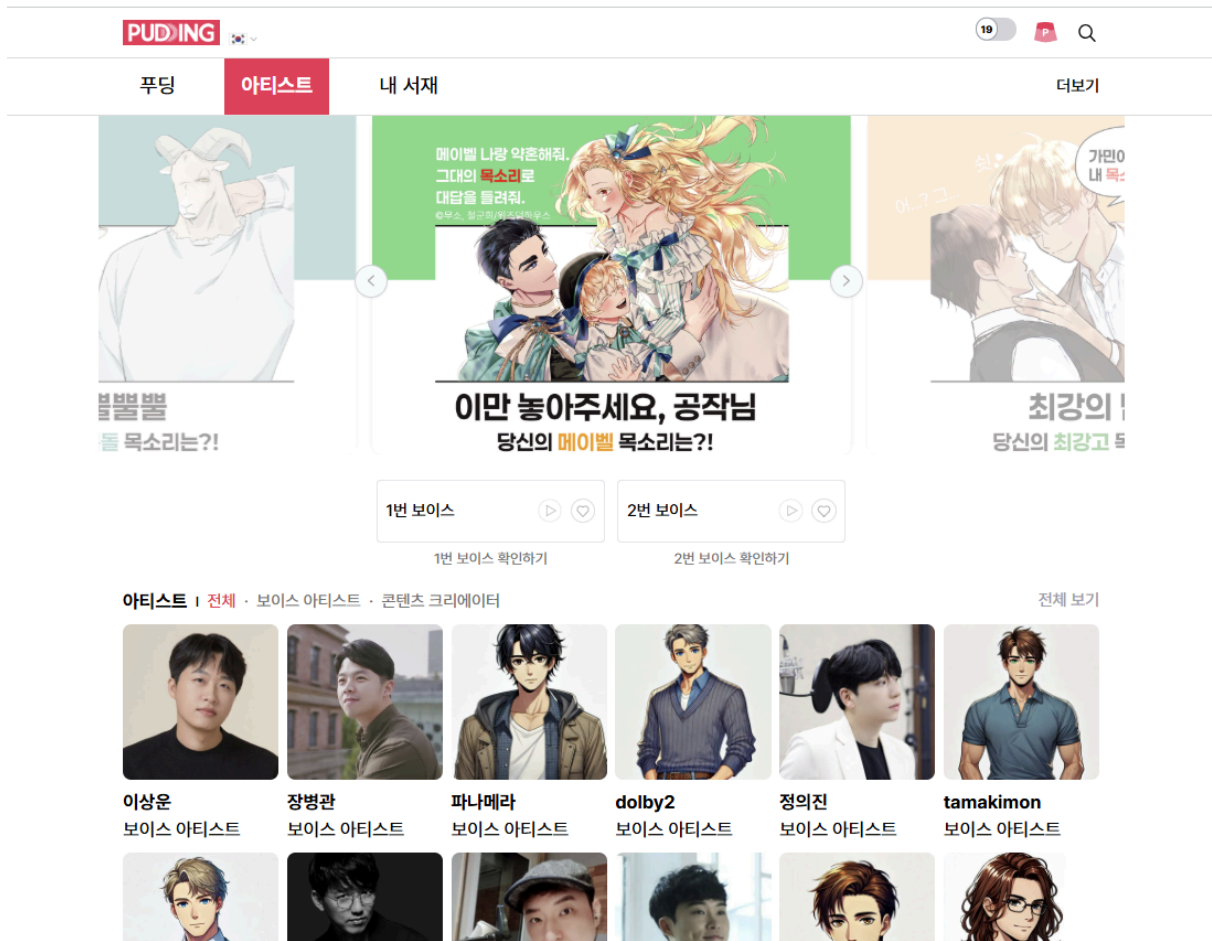


▶
PLAY

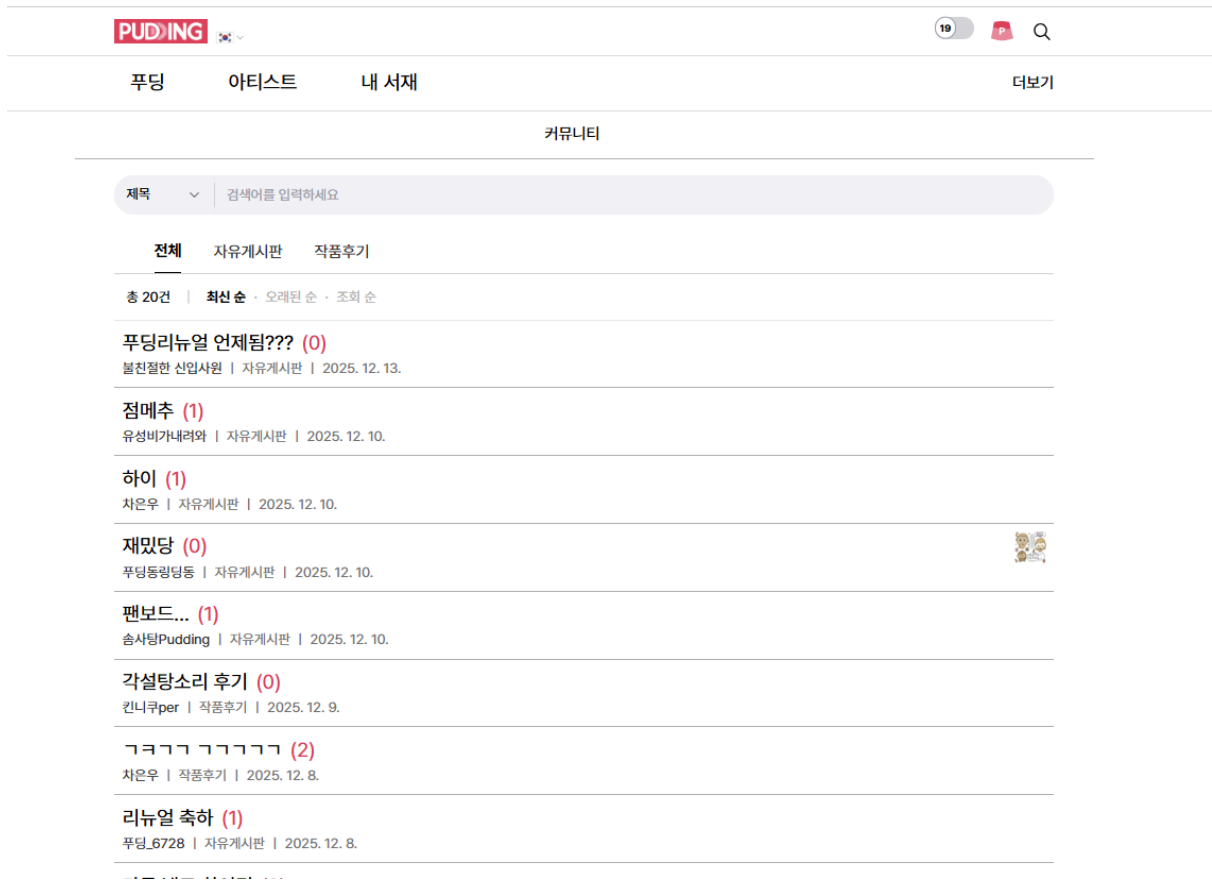




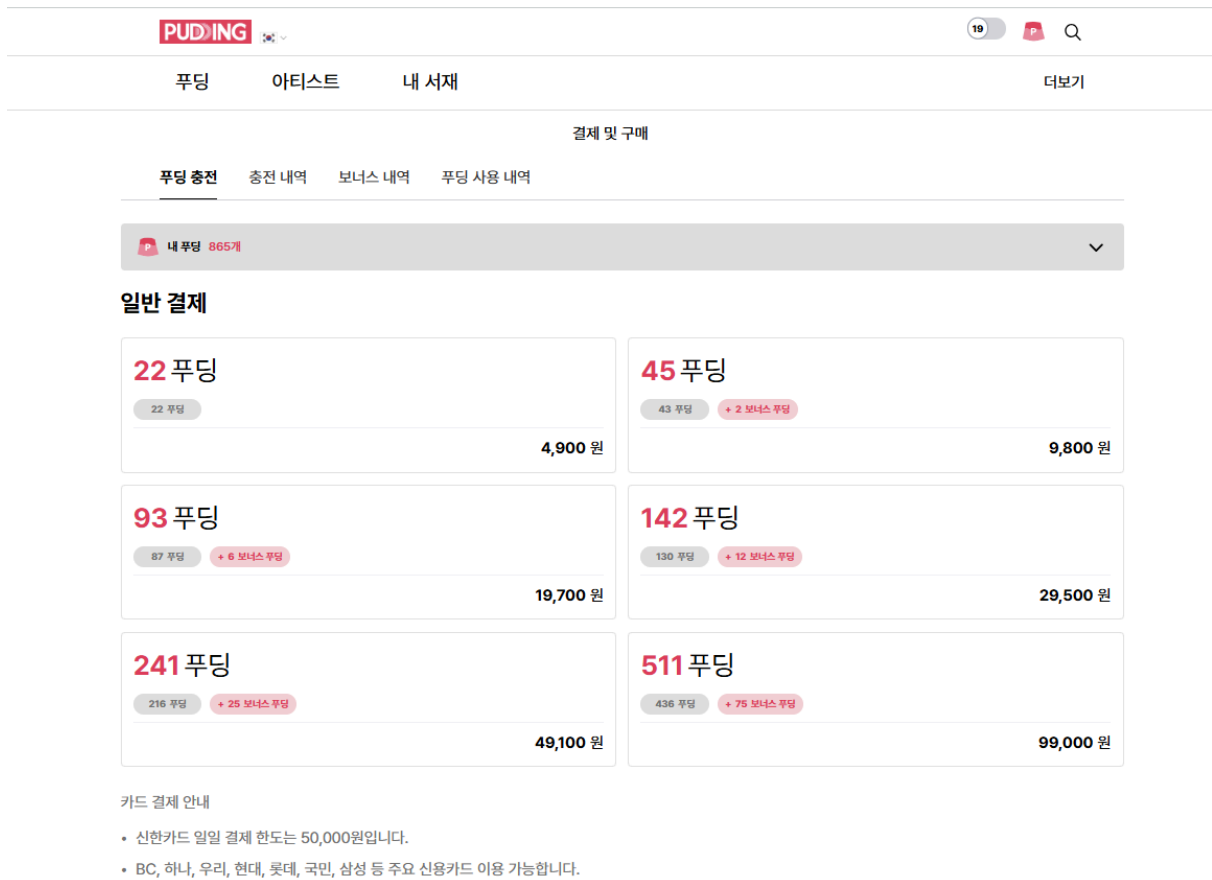
이미지 3. 플레이어 페이지



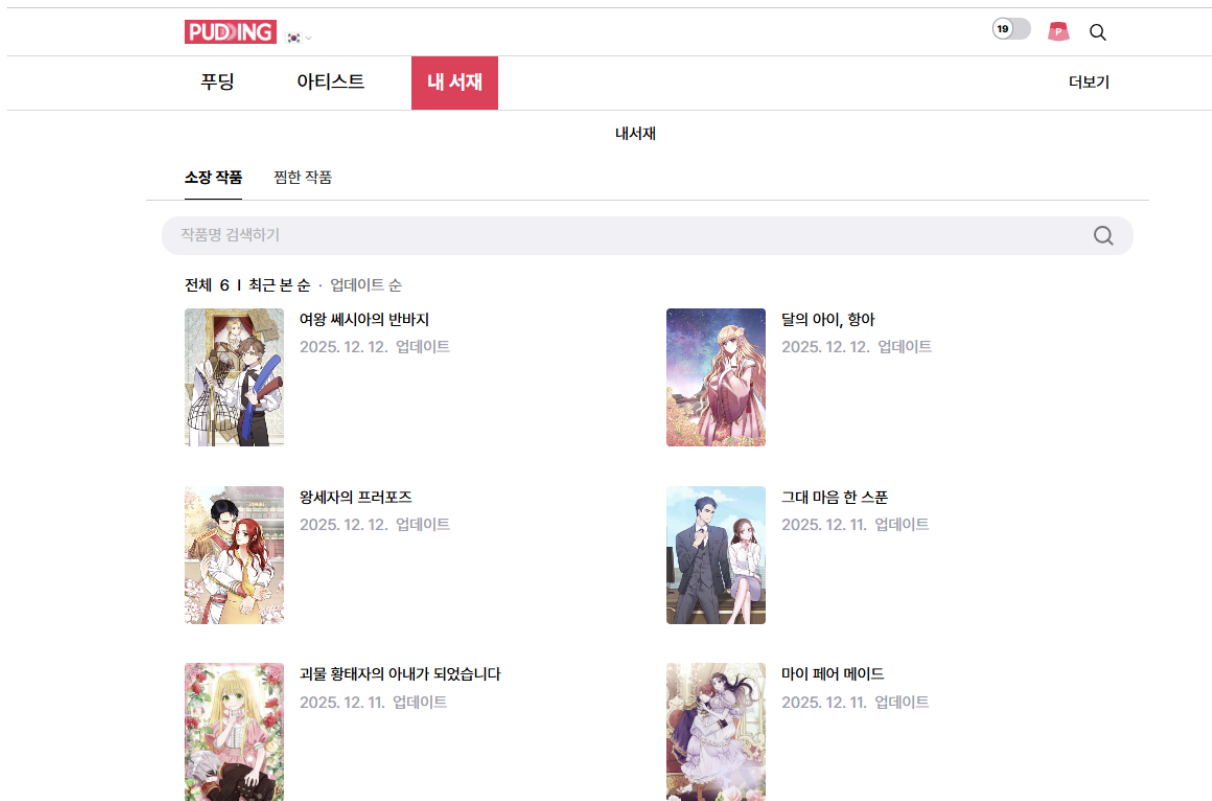
이미지 4. 아티스트 메인 페이지



이미지 5. 커뮤니티 페이지



이미지 6. 결제 및 구매 페이지



이미지 7. 내 서재 페이지

'
 '
 '